

Tarea C — Recuperación léxica contra semántica latente sobre un corpus mínimo

MMIA 6013 · Taller 03 v2 · reporte.pdf

Objetivo

Comparar dos recuperadores sobre un corpus mínimo de 10 documentos y 6 consultas (cada una con un documento relevante): (1) la línea base léxica TF-IDF con similitud coseno y (2) la semántica latente LSA, que proyecta la misma matriz TF-IDF a 4 dimensiones con TruncatedSVD. Se evalúan Hit@1, Hit@3 y MRR, se analizan los fallos por consulta mediante el solapamiento léxico consulta–documento relevante y se discute qué aportaría un recuperador denso y a qué costo.

Método

- **Datos.** Corpus d01–d10 y consultas q1–q6 literales del enunciado, con juicios q1→d04, q2→d03, q3→d08, q4→d05, q5→d09, q6→d02.
- **TF-IDF.** `TfidfVectorizer` de `scikit-learn` con todos los parámetros por defecto (vocabulario resultante: 105 términos); las consultas se transforman con el mismo vectorizador; similitud coseno consulta–documento y orden descendente.
- **LSA.** `TruncatedSVD(n_components=4, random_state=0)` ajustado sobre la matriz TF-IDF del corpus (varianza explicada acumulada: 0.39534217090869533); documentos y consultas proyectados al espacio latente de 4 dimensiones; similitud coseno en ese espacio.
- **Evaluación.** Sobre las 6 consultas: Hit@1 y Hit@3 (fracción de consultas con el relevante en el puesto 1 o en el top 3) y MRR (media de 1/rank del relevante). Fallo = relevante fuera del top 3.

```
vect = TfidfVectorizer() # parámetros por defecto
X = vect.fit_transform(docs) # 10 x 105 términos
Q = vect.transform(queries)
S = cosine_similarity(Q, X) # ranking TF-IDF + coseno
svd = TruncatedSVD(n_components=4, random_state=0)
Xl, Ql = svd.fit_transform(X), svd.transform(Q)
Sl = cosine_similarity(Ql, Xl) # ranking LSA + coseno
# por consulta: orden descendente de S[i]; Hit@1/Hit@3 y MRR = media de 1/rank
```

(Extracto; el script completo acompaña la entrega.)

Resultados

Tabla 1 (Parte 2). Métricas de los dos métodos sobre las 6 consultas.

Método	Hit@1	Hit@3	MRR
TF-IDF (léxico)	0.8333 (5/6)	0.8333 (5/6)	0.8611
LSA (4 dimensiones)	0.6667	0.8333	0.7500

Tabla 2 (Parte 3). Rank del relevante por consulta y términos compartidos entre la consulta y su documento relevante.

Consulta (relevante)	Rank TF-IDF	Rank LSA	Términos compartidos (n)
q1 (d04)	1	1	8: de, frecuencia, función, la, léxica, pondera, ranking, términos
q2 (d03)	1	1	3: al, fragmentos, prompt
q3 (d08)	1	1	5: bajo, de, entrena, matrices, rango
q4 (d05)	1	3	4: compara, la, orientación, vectores
q5 (d09)	6	6	1: la
q6 (d02)	1	1	5: autoatención, capas, con, feed, forward

Fallos (relevante fuera del top 3). TF-IDF falla únicamente q5: d09 aparece en el puesto 6 con

sim=0.0790, por detrás de d03 (0.3107) y d06 (0.2009). LSA también falla solo q5 (d09 en el puesto 6, sim=0.4999) y, además, pierde el Hit@1 en q4: d05 baja al puesto 3 (sim=0.8984), superado por d01 (0.9220) y d10 (0.9206).

Discusión

Por qué falla q5 en ambos métodos. Las consultas que ambos métodos resuelven comparten términos de contenido con su relevante (q1: 8 términos; q3 y q6: 5; q4: 4; q2: 3, con "fragmentos" y "prompt"). En q5, en cambio, la consulta ("determinista", "elegir", "siguiente palabra") y d09 ("temperatura", "logits", "softmax", "probabilidad") comparten un único término: "la". Sin solapamiento de contenido, TF-IDF solo puede puntuar d09 por "la" (sim=0.0790) y lo superan documentos que comparten palabras funcionales con la consulta, como d03 (por "modelo") y d06 (por "qué", "el"). LSA no corrige el fallo: sus 4 factores latentes se aprenden de la co-ocurrencia de estos 10 documentos, no de un lenguaje más amplio, por lo que no pueden acercar "determinista" a "temperatura"; la proyección además difumina q4 (d05 pasa del puesto 1 al 3).

Parte 4 — ¿por qué un recuperador denso resolvería q5 mejor? Porque sus embeddings provienen de un modelo entrenado sobre grandes corpus: codifican proximidad semántica y paráfrasis, de modo que "más determinista al elegir la siguiente palabra" quedaría cerca de "temperatura ... valores bajos concentran la probabilidad" aunque no compartan términos de contenido (solo "la"). El material del curso lo resume: el retrieval denso "entiende semántica, sinónimos, paráfrasis" (s2-rag-y-vector-search.md, §5.2), mientras que el léxico "no entiende sinónimos ni paráfrasis" (s2-rag-y-vector-search.md, §5.1). LSA no hereda esa capacidad porque su "entrenamiento" se limita a la co-ocurrencia de estos 10 documentos.

Costos frente a TF-IDF: exige un modelo preentrenado (memoria para el encoder), cómputo y latencia para codificar consultas y documentos, entrenamiento o ajuste del encoder al dominio, menor interpretabilidad y puede fallar en términos técnicos exactos, IDs y números específicos (s2-rag-y-vector-search.md, §5.2); TF-IDF, en cambio, es ligero, no requiere entrenamiento ni GPU y es explicable, pues cada puntuación se traza a términos compartidos.